

High-Performance C++ Dual-Pivot Quicksort

A Drop-in Replacement for `std::sort`

Lin Zhanzhi

Final Year Project — Final Presentation

2026

- 1 **The Product** — what `dpqs::sort` is and what it delivers
- 2 **Research Gap & Methodology** — why another sort, how we measured
- 3 **Sequential Walkthrough** — seven guards, in execution order
 - Early termination · type dispatch
 - Insertion leaves · run-merging · heapsort fallback
 - Dual-pivot · Dutch National Flag
- 4 **Parallel Path** — work-stealing, scaling curve
- 5 **VTune Root-Cause** — why 16T regresses
- 6 **Engineering, Limitations, Conclusion**
- 7 **Q & A**

- 1 **The Product**
- 2 Research Gap & Methodology
- 3 Sequential Walkthrough
- 4 Parallel Path
- 5 VTune Root-Cause
- 6 Engineering, Limitations, Conclusion
- 7 Q & A



The Product: `dpqs::sort(...)`

What it is

- Header-only C++17 sorting library
- Drop-in replacement for `std::sort`
- Same iterator + comparator contract

What it delivers

- = parity on random 32-bit int
- $\sim 10\times$ on reverse-sorted (near-linear)
- $4.72\times$ peak parallel speedup
- Zero dependencies — one `#include`

```
#include "dual_pivot_quicksort.hpp"
```

```
std::vector<int> v = {...};
```

```
// Auto-parallel (default)
```

```
dual_pivot::sort(v);
```

```
// Iterator form
```

```
dual_pivot::sort(v.begin(), v.end())
```

```
// Force sequential
```

```
dual_pivot::sort(v, 1);
```

```
// Custom comparator
```

```
dual_pivot::sort(v, std::greater<int>)
```

- 1 The Product
- 2 **Research Gap & Methodology**
- 3 Sequential Walkthrough
- 4 Parallel Path
- 5 VTune Root-Cause
- 6 Engineering, Limitations, Conclusion
- 7 Q & A



Research Gap — Why Another Sort?

The Java-C++ discrepancy

- Java adopted Yaroslavskiy's dual-pivot quicksort in **JDK 7 (2011)** for primitive types
- C++ `std::sort` (libstdc++, libc++, MSVC) still uses **single-pivot Introsort** [3]
- No widely adopted, STL-compliant dual-pivot implementation exists in production C++

Does the Java win translate to C++?

- Java: expensive comparisons → "more comparisons, fewer memory accesses" is a clear win
- C++: templates inline comparators → comparisons are near-free
- **Open question:** does the memory-traffic reduction survive?

What this project does

- 1 **Engineer** a modern, STL-compliant C++17 dual-pivot sort
- 2 **Measure** whether it can beat `std::sort` in a native environment
- 3 **Extend** it with adaptive run-merging, DNF, and work-stealing parallelism

Existing C++ DPQS code is non-generic, experimental, or pre-C++11 — not production-grade.

Hardware

- Intel Core i5-12600KF
- 6 P-cores (SMT) + 4 E-cores — **10 physical / 16 logical**
- L1-D: 6×48 KB (P) + 4×32 KB (E)
- L1-I: 6×32 KB (P) + 4×64 KB (E)
- L2: 6×1.25 MB (P) + 2×2 MB (E-cluster)
- **L3: 20 MB** shared
- 32 GB DDR5 @ 2992.7 MHz

Compiler

- GCC 13.3.0 (MinGW-w64)
- `-std=c++17 -O2 -march=native -DNDEBUG`
- `-pthread`

Workload — 7 872 configurations

- **6 algorithms** — `std::sort`, `dual_pivot_sequential`, `dual_pivot_parallel_{2,4,8,16}`
- **4 data types** — `int`, `int8_t`, `int16_t`, `double`
- **8 patterns** — random, nearly-sorted, reverse-sorted, organ-pipe, sawtooth, many-duplicates {10, 50, 90}%
- **41 sizes** — log-spaced $10^3 \rightarrow 10^7$

Randomness

- 10 seeds (42...51)
- **3 warmups + 10 timed** per seed
- Representative = **median of per-seed minima** (100 samples)

Structured patterns

- 30 iterations, min-time

Section 3 — Sequential Walkthrough

- 1 The Product
- 2 Research Gap & Methodology
- 3 **Sequential Walkthrough**
- 4 Parallel Path
- 5 VTune Root-Cause
- 6 Engineering, Limitations, Conclusion
- 7 Q & A



Execution Flow — What Happens Inside `dpqs::sort(...)` [1]

```
sort(container | iter,iter | ptr,len)
  |
  v
[Step 0] Normalize API overloads
        -> sort(T* a, parallelism, lo, hi, Compare)
        |
        v
[Step 1] Guards: null / range / early-termination scan --> DONE (sorted)
        |
        v
[Step 2] Type dispatch (if constexpr)
        |--- int8/int16 -----> counting_sort()           --> DONE
        |--- float/double -----> sort_floats()
        |--- parallel eligible --> parallelQuickSort()     --> §3
        |--- otherwise -----> sort_sequential()
```

Steps 3–5 continue on the next slide.

Execution Flow (cont.) — Steps 3, 4, 5

[Step 3] `Compute max_depth = 2 * log2(n) * DELTA`
 `(introsort safety net)`

|
v

[Step 4] `Recursive core – guards 4.1 ... 4.7`
 `(walked in detail on next slides)`

|
v

[Step 5] `Return (sorted in place)`

We walk the numbered steps in execution order, and show the benchmark number each step earns.

Step 1 — Early Termination (Free $O(n)$ Win)

Guard: single linear scan before any sorting work

- `low >= high` → return
- Null check / range validation
- **checkEarlyTermination:** $O(n)$ scan; if already sorted → done

Result — fully-sorted input, 10^7 int

<code>std::sort</code>	≈ baseline (still partitions)
<code>dpqs</code>	near-instant ($O(n)$)

Takeaway

The cheapest guard wins the easiest case outright.

"Data is already sorted" is more common in production than people think — log ingestion, prefix arrays, time-series append.

Step 2 — Type Dispatch (Compile-Time if constexpr)

Type	Strategy	Complexity
int8_t, int16_t	Counting sort [6]	$O(n + k)$
float, double	sort_floats (NaN / ± 0 aware)	$O(n \log n)$
Parallel-eligible	parallelQuickSort (§3)	$O(n \log n / p)$
Otherwise	Dual-pivot quicksort [2]	$O(n \log n)$

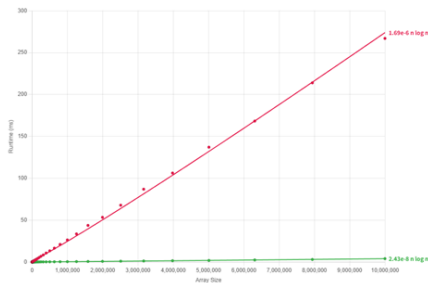


Figure 1: int8_t — dpqs vs std::sort

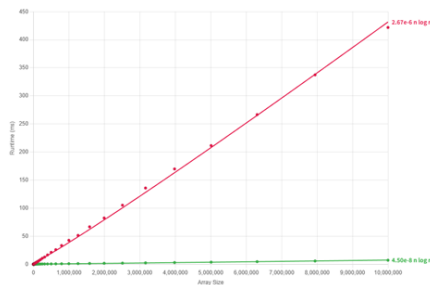


Figure 2: int16_t — dpqs vs std::sort

Step 4 — Recursive Core: Seven Guards, in Order

```
sort_sequential(lo, hi, leftmost):  
    size = hi - lo  
    if size < 65 and !leftmost          -> 4.1  mixed (pin) insertion sort  
    if size < 45 and leftmost           -> 4.2  plain insertion sort  
    if size > MIN_TRY_MERGE_SIZE:  
        if try_merge_runs(...)         -> 4.3  Timsort-style run merge  <DONE>  
    if depth >= max_depth               -> 4.4  heapsort fallback [3]   <DONE>  
  
    # --- pivot machinery ---  
    sort5_network(a[e1..e5])            -> 4.5  9-comparator network  
    if a[e1] < a[e2] < a[e3] < a[e4] < a[e5]:  
        dual_pivot_partition(P1=e1, P2=e5) -> 4.6a 3 regions  
    else:  
        dnf_partition(P=e3)             -> 4.6b [<P][=P][>P]  
    # --- recurse (4.7): loop on largest, recurse on two smaller ---
```

Each guard is shown next with the input pattern where it dominates.

4.1 / 4.2 — Insertion-Sort Leaves (Classical)

What & why

- size < 65 (non-leftmost) → **mixed/pin insertion**; left pivot is a sentinel so no bound check
- size < 45 (leftmost) → **plain insertion**
- Every leaf of the quicksort tree hits one of these

Result — random int, 10^7

Algorithm	Speedup
<code>std::sort</code>	baseline
<code>dpqs</code>	$\approx 1.1\times$

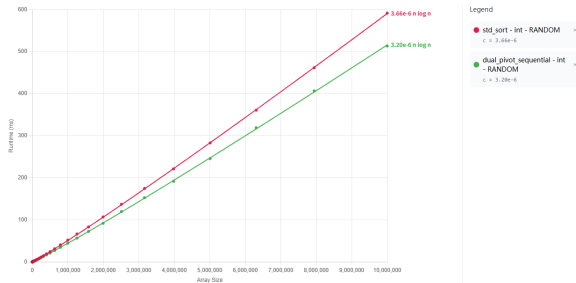


Figure 3: Random int32 — dpqs vs std::sort

4.3 — Adaptive Run-Merging (Timsort Lineage [4])

Scan for ascending / descending / constant runs

- If first run $< \text{MIN_FIRST_RUN_SIZE}$ → **bail out fast** (random data pays tiny cost)
- Else merge runs in $O(n \log r)$ — **no partitioning, no recursion spawned**

r = number of monotone runs detected in the input
($1 \leq r \leq n/2$). Nearly-sorted $\Rightarrow r$ small $\Rightarrow$ near-linear.

Pattern	Speedup
Nearly-sorted	$\approx 0.74\times$ (slower)
Reverse-sorted	$\approx 10\times$ (near-linear)
Organ-pipe / Sawtooth	see figures

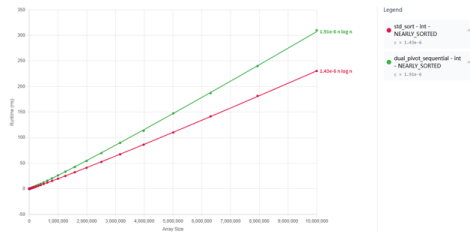


Figure 4: Nearly-sorted

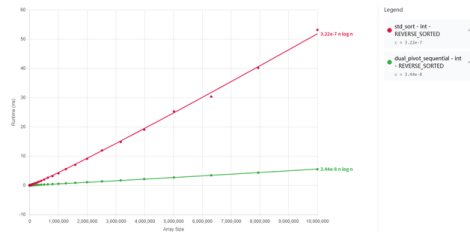


Figure 5: Reverse-sorted

4.3 (cont.) — Organ-Pipe & Sawtooth

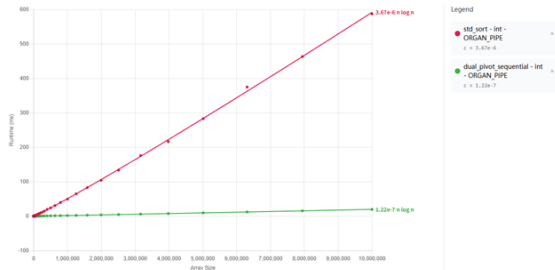


Figure 6: Organ-pipe

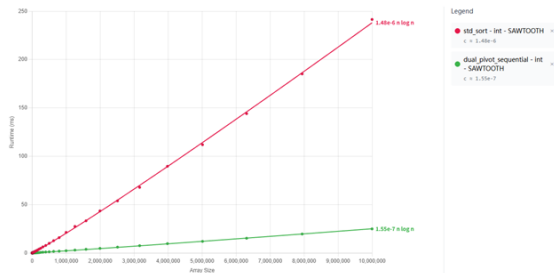


Figure 7: Sawtooth

One forward scan detects the runs; one merge pass fuses them.
Full sort collapses to linearithmic work in $r = \text{\#runs}$, often $r \ll \log n$.

4.5 / 4.6 — Pivots From a 9-Comparator Network

Pick 5 samples anchored to the two ends: **4.6a — Dual-pivot partition** [2]

```
step = (size / 8) * 3 + 3    // ~ 3/8 size
e1   = low  + step          // ~ 3/8 point
e5   = high - step          // ~ 5/8 point
e3   = (e1 + e5) / 2        // middle
e2   = (e1 + e3) / 2
e4   = (e3 + e5) / 2
```

[< P1][P1 <= x <= P2][> P2]
P1 = a[e1], P2 = a[e5]
backward scan + prefetch

Samples sit inside the central quarter,
giving more balanced pivots.

Sort with hardcoded network —

sort5_network = 9 comparators,
branch-free.

Then branch on ordering:

```
if a[e1] < a[e2] < a[e3]
    < a[e4] < a[e5]:
    # strictly ordered
    DUAL-PIVOT (4.6a)
```

4.6b — Dutch National Flag [5]

```
[ < P ][ = P ][ > P ]
P = a[e3]
middle region excluded from recursion
```

4.6 — Duplicates Route to DNF Automatically

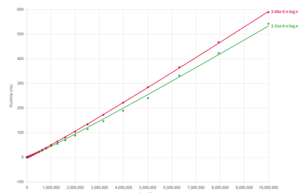


Figure 8: 10% unique

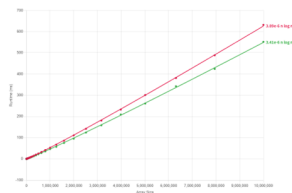


Figure 9: 50% unique

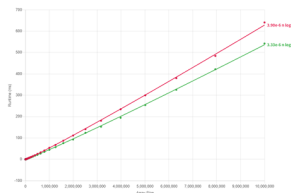


Figure 10: 90% unique

Absolute vs std::sort (int, $n = 10^7$): $1.09\text{--}1.18\times$ across the whole duplicate range — widest margin at *light* duplication, not heavy.

Why no huge win over our own random baseline? The input is still randomly permuted, so (i)~run-merging bails out — constant sub-sequences are length~1, (ii)~the 5-sample oracle only flags duplicates probabilistically, firing reliably once sub-arrays shrink enough that 5 random draws collide, (iii)~each DNF call absorbs only one pivot-value's band — typically 10–20% of the sub-problem.

Why we still win against std::sort. `libstdc++` is 2-way introsort with no DNF. Heavy duplicates luckily centre its pivots, so it stays competitive there; at moderate duplication its pivot quality degrades and our DNF keeps a steady edge.

Sequential Summary — One Table, Every Pattern

Pattern	Speedup	Responsible guard
Already sorted	near-instant	Early termination (Step 1)
int8_t / int16_t random	$\sim 8\times$ / $\sim 6\times$	Counting-sort dispatch (Step 2)
Random int	$\sim 1.0\times$ (parity)	Insertion-sort leaves (4.1 / 4.2)
Nearly-sorted	$\approx 0.74\times$ (slower)	try_merge_runs bails out (4.3)
Reverse-sorted	$\approx 10\times$, near-linear	try_merge_runs (4.3)
Many duplicates (any mix)	$1.09\text{--}1.18\times$	DNF partition (4.6b)

Every number above comes from the same hardware, same 10^7 elements, same methodology.

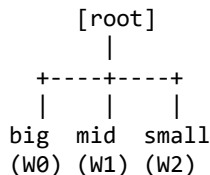
Section 4 — Parallel Path

- 1 The Product
- 2 Research Gap & Methodology
- 3 Sequential Walkthrough
- 4 **Parallel Path**
- 5 VTune Root-Cause
- 6 Engineering, Limitations, Conclusion
- 7 Q & A



Parallel Path — Work-Stealing Thread Pool

Partition tree



Worker dequeues

W0: [_ | _ | _]
W1: [_ | _]
W2: []
W3: [_]

<- idle

Worker W2 is idle:
-> steals tail from W0

Rules

- Push two larger sub-ranges to own deque; keep smallest (loop)
- Idle worker steals from another deque's tail (sticky-victim)
- MIN_PARALLEL_SORT_SIZE cutoff prevents micro-tasks

Start-up (few tasks, many idle workers)

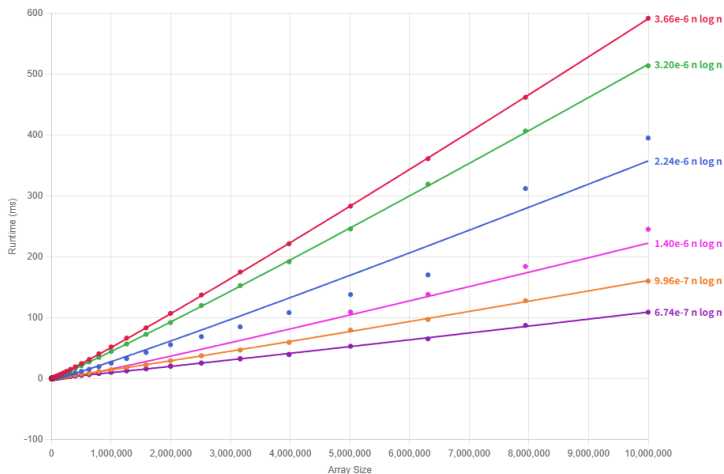
At launch only the root partition exists — 1 task, N-1 idle workers. Idle workers consult a global “done” flag; while

Scaling — 10^7 random int

T	Time (ms)	Speedup	Eff.
1	513.4	1.00×	100 %
2	394.7	1.30×	65 %
4	159.8	3.21×	80 %
8	108.7	4.72×	59 %
16	244.9	2.10×	13 %

Peak at 8T; 16T regresses (see VTune slide for root cause).

Scaling Chart + the 8T Peak



Legend

- std_sort - int - RANDOM
 $c \approx 3.66e-6$
- dual_pivot_sequential - int - RANDOM
 $c \approx 3.20e-6$
- dual_pivot_parallel_2 - int - RANDOM
 $c \approx 2.24e-6$
- dual_pivot_parallel_4 - int - RANDOM
 $c \approx 9.96e-7$
- dual_pivot_parallel_8 - int - RANDOM
 $c \approx 6.74e-7$
- dual_pivot_parallel_16 - int - RANDOM
 $c \approx 1.40e-6$

Figure 11: 10^7 random int — std::sort vs dpqs at 1 / 2 / 4 / 8 / 16 threads

Near-linear through 4T → sweet spot at **8T (4.72×)** → **regression at 16T.**

- 1 The Product
- 2 Research Gap & Methodology
- 3 Sequential Walkthrough
- 4 Parallel Path
- 5 **VTune Root-Cause**
- 6 Engineering, Limitations, Conclusion
- 7 Q & A



VTune Root-Cause — Four Stacked Ceilings (8T vs 16T)

Metric	8T	16T	Δ	Diagnosis
L3-Bound (% ticks)	14.5%	25.1%	+10.6 pp	(1) Working set overflows 20 MB L3
Machine Clears (% slots)	1.8%	7.5%	$\times 4.2$	(3) SMT memory-order nukes
L2-Bound (% ticks)	0.0%	1.0%	+1.0 pp	(3) SMT siblings share L2
L1-Bound (% ticks)	13.8%	14.7%	+0.9 pp	(3) SMT siblings share L1D
Slow-Pause spin-wait	0.0%	0.8%	+0.8 pp	(4) Mutex/steal contention
E-core clockticks	10.9G	29.8G	$\times 2.7$	(2) Work spilled to slower cores
E-core CPI	1.37	1.37	—	E-core $\approx 20\%$ slower / instr than P-core (1.14)
P-core CPI	1.14	1.55	+36%	Net: every instr pays more cycles
DRAM-Bound	0.6%	0.5%	≈ 0	Bandwidth is NOT the bottleneck

Four stacked effects: **(1)** L3 latency ; $\gg$; **(2)** hybrid-core dilution ; $>$; **(3)** SMT cache sharing ; $>$; **(4)** mutex contention.

The silicon is the ceiling — not our algorithm, not our thread pool.

Section 6 — Engineering, Limitations, Conclusion

- 1 The Product
- 2 Research Gap & Methodology
- 3 Sequential Walkthrough
- 4 Parallel Path
- 5 VTune Root-Cause
- 6 **Engineering, Limitations, Conclusion**
- 7 Q & A



- **Header-only, zero deps** — one `#include`, any C++17 build
- **Fully generic templates** — `T`, iterator category, Compare all type-parameters
- **STL-compatible iterator adapter**
 - contiguous → raw-pointer fast path (zero-cost)
 - non-contiguous → temporary buffer
- **if constexpr compile-time dispatch** — counting / float / dual-pivot decided at compile time
- **Hardware hints** — `DPQS_PREFETCH_READ` wraps `__builtin_prefetch` on the partition hot loop

Modular headers

```
include/  
dual_pivot_quicksort.hpp    (façade)  
dpqs/  
    sequential_sorters.hpp  
    run_merger.hpp  
    counting_sort.hpp  
    thread_pool.hpp  
    utils.hpp
```

Honest Limitations

- **Not stable** — like `std::sort`, unlike `std::stable_sort`
- **Move-only types** — run-merger needs a copy path
- **Nearly-sorted regression** at some sizes — threshold tuning WIP

Reflection

The biggest lesson wasn't the algorithm — it was that the real engineering is the **7 800-configuration harness + multi-seed methodology** that makes every tuning decision defensible.

Delivered

- Header-only, STL-compatible DPQS for C++17
- Adaptive: counting sort · run-merger · DNF · introsort fallback
- Work-stealing parallel — **4.72**× peak
- VTune-validated root-cause for the scaling curve

What this FYP taught me

- Tuning a program for a **real machine** is as much an exercise in *silicon literacy* as in algorithms — caches, SMT, hybrid cores all shape the curve.
- Every optimisation is a **trade-off**: a win on one pattern is usually a loss somewhere else. The honest $0.74\times$ on nearly-sorted is the price of $10\times$ on reverse-sorted.
- The **more specific** the target scenario, the easier tuning becomes — and the easier a trade-off is to accept.

Still a long way to go

- Generality *not* fully proven — the implementation is tuned against one CPU, one compiler, one workload matrix.
- Cross-platform, cross-compiler, and cross-architecture validation is the next milestone.
- Measure-first, tune-second is a discipline I will carry forward.

The repository gave me my first taste of real performance engineering.

- 1 The Product
- 2 Research Gap & Methodology
- 3 Sequential Walkthrough
- 4 Parallel Path
- 5 VTune Root-Cause
- 6 Engineering, Limitations, Conclusion
- 7 **Q & A**



Questions?

Thank you for your attention.

References

- [1] C. A. R. Hoare, "Quicksort," *The Computer Journal*, vol. 5, no. 1, pp. 10–16, 1962.
- [2] V. Yaroslavskiy, "Dual-pivot quicksort algorithm," Research report, 2009. [Online]. Available: <http://codeblab.com/wp-content/uploads/2009/09/DualPivotQuicksort.pdf>
- [3] D. R. Musser, "Introspective sorting and selection algorithms," *Software: Practice and Experience*, vol. 27, no. 8, pp. 983–993, Aug. 1997.
- [4] T. Peters, "Timsort," CPython `listsort` description, 2002. [Online]. Available: <https://github.com/python/cpython/blob/main/Objects/listsort.txt>
- [5] E. W. Dijkstra, *A Discipline of Programming*. Englewood Cliffs, NJ, USA: Prentice-Hall, 1976.
- [6] H. H. Seward, "Information sorting in the application of electronic digital computers to business operations," M.S. thesis, MIT, Cambridge, MA, USA, 1954.
- [7] R. D. Blumofe and C. E. Leiserson, "Scheduling multithreaded computations by work stealing," *Journal of the ACM*, vol. 46, no. 5, pp. 720–748, Sep. 1999.
- [8] D. Chase and Y. Lev, "Dynamic circular work-stealing deque," in *Proc. 17th ACM Symp. Parallelism in Algorithms and Architectures (SPAA)*, 2005, pp. 21–28.
- [9] M. Aumüller and M. Dietzfelbinger, "Optimal partitioning for dual-pivot quicksort," *ACM Transactions on Algorithms*, vol. 12, no. 2, art. 18, Feb. 2016.

```
// e1 e2 e3 e4 e5 (indices into the array)
1:  (e1,e2)  if a[e1]>a[e2] swap
2:  (e4,e5)  if a[e4]>a[e5] swap
3:  (e3,e5)  if a[e3]>a[e5] swap
4:  (e3,e4)  if a[e3]>a[e4] swap
5:  (e2,e5)  if a[e2]>a[e5] swap
6:  (e1,e4)  if a[e1]>a[e4] swap
7:  (e1,e3)  if a[e1]>a[e3] swap
8:  (e2,e4)  if a[e2]>a[e4] swap
9:  (e2,e3)  if a[e2]>a[e3] swap
```

Properties

- Proven optimal: 9 is the minimum comparator count to sort 5 elements
- Data-dependency depth = 6 → pipelines well on modern OoO CPUs
- Branch-free conditional swap via `std::min` / `std::max` (`cmov`)

Backup — Multi-Seed Protocol (Why Median-of-Medians?)

```
for seed in {42, 43, ..., 51}:                                # 10 seeds
    data = generate_random(size, seed)
    for w in 1..3: sort(copy_of(data))                          # warmup
    times = []
    for i in 1..10: times.push(time(sort(copy_of(data))))
    per_seed_median = median(times)                             # absorbs run jitter
medians = [...]                                                # 10 per-seed medians
representative = median(medians)                               # absorbs one unlucky seed
```

- **Mean across seeds** → skewed by one bad permutation
- **Min across seeds** → rewards one lucky permutation
- **Median-of-medians** → robust to both; aligns with current HPC benchmarking practice

Backup — Full Hardware Spec (from CPU-Z)

CPU — Intel Core i5-12600KF

Field	Value
Generation	12th Gen, Alder Lake
Socket	LGA1700
Process	10 nm
Cores / threads	10 (6P + 4E) / 16
Max TDP	125 W
Core voltage	1.172 V
Core #0 speed	\$□\$4489 MHz
Multiplier	\$×\$45.0 (range 48.0–49.0)
Bus speed	99.76 MHz

Caches

Level	Size
L1-D	6 × 48 KB + 4 × 32 KB
L1-I	6 × 32 KB + 4 × 64 KB
L2	6 × 1.25 MB + 2 × 2 MB

Memory — 32 GB DDR5

Field	Value
Type	DDR5
Size	32 GB
Channels	4 × 32-bit (dual-channel)
DRAM frequency	2992.7 MHz (eff. \$□\$5985 MT/s)
Memory ctrl freq	1496.3 MHz
LLC / Ring	3591.2 MHz

Primary Timings

Param	Clocks
CAS Latency (CL)	36
tRCD	38
tRP	38
tRAS	80
tRC	118
Command Rate	2T